Final Report – AI Cloud Album

Group 05: Shi He, Huaitang Wang, Boxiang Zhang

Project Overview

The AI Cloud Album is a cloud-based application designed to help users efficiently manage and explore large collections of images. Traditional photo management methods rely heavily on manual organization, which becomes inefficient and time-consuming as the number of images grows. This project addresses that problem by introducing an intelligent system that automatically classifies images, extracts metadata such as location, and organizes photos into searchable collections.

The target audience includes users who frequently capture or store large numbers of photos, such as travelers, photographers, and everyday smartphone users. These users often struggle to organize and retrieve specific images from their collections. The AI Cloud Album provides a more efficient and user-friendly solution by combining automation with user interaction.

The system offers several key functionalities. Users can upload individual images or ZIP files through a web interface and store them in cloud storage. Once uploaded, images are processed asynchronously using cloud services, where a machine learning model classifies the image content and extracts relevant metadata. Based on the classification results, the system may generate follow-up questions to collect additional information from the user, improving search accuracy and personalization.

Users can browse their images by category and location. The system also supports map-based visualization, allowing users to view where their photos were taken. Additional features include user authentication and batch download of images.

Overall, the AI Cloud Album demonstrates a complete cloud-native workflow that integrates frontend interaction, backend APIs, scalable storage, asynchronous processing, and machine learning. It provides a practical solution for intelligent photo management while showcasing modern cloud computing architecture.

Architecture and Implementation

System Architecture

The AI Cloud Album is designed as a cloud-native, distributed system that integrates frontend interaction, backend services, cloud storage, asynchronous processing, and machine learning

components. The overall architecture follows a decoupled design to ensure scalability, responsiveness, and maintainability.

As shown in *Figure 1*, the system consists of five main components: the frontend web application, the Flask backend API, the asynchronous processing layer (SQS and Lambda), the data storage layer (Amazon S3 and DynamoDB), and the machine learning and external service integration.

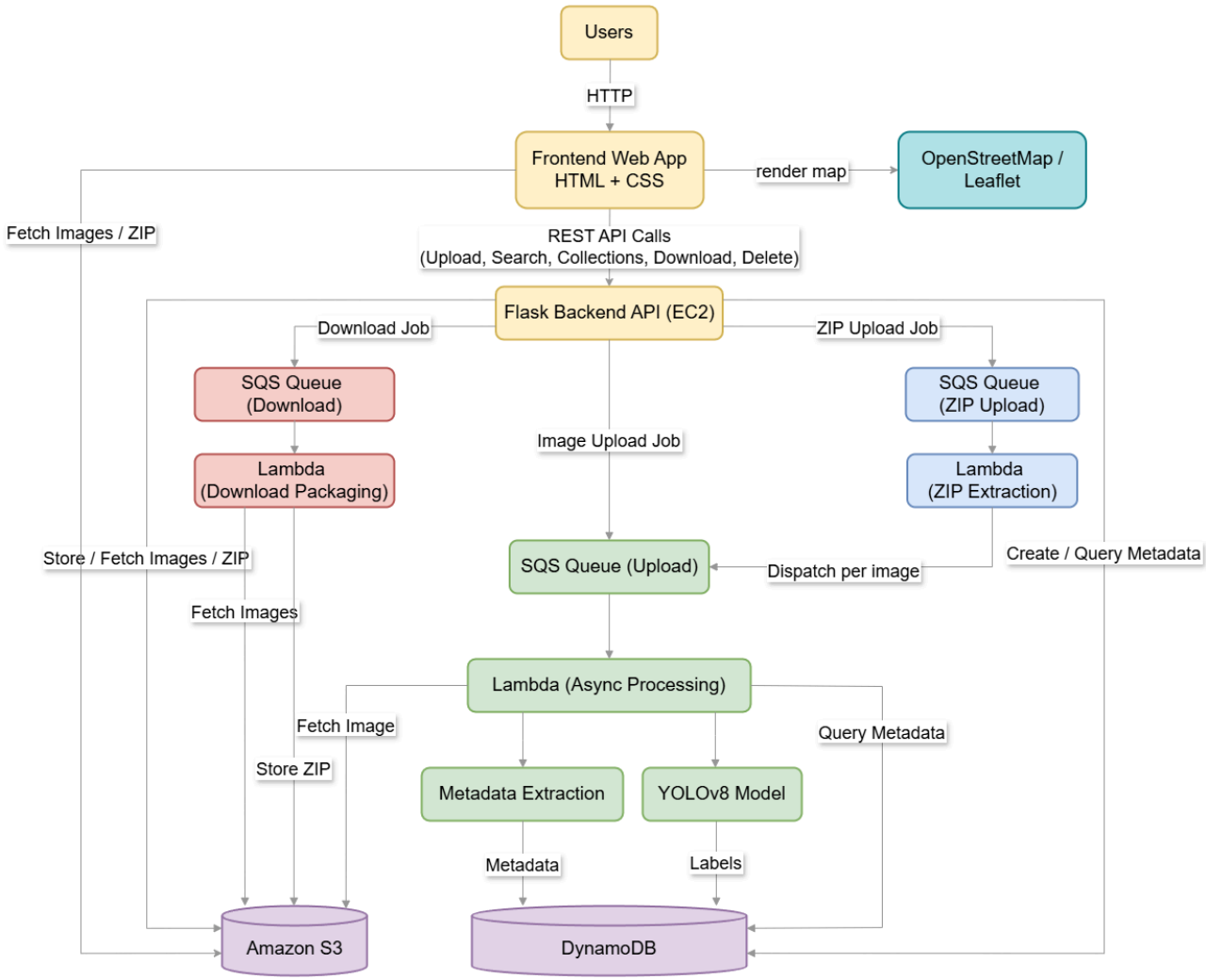


Figure 1. System Architecture Diagram

Frontend Web Application

The frontend is implemented using HTML and CSS and serves as the user interface of the system. It allows users to register and log in, upload images or ZIP files, browse uploaded images, view classification results, and interact with map-based visualization.

The frontend communicates with the backend via RESTful APIs over HTTP. It also periodically polls the backend to retrieve the status of asynchronous tasks such as image processing.

Flask Backend API

The backend is built using Python and Flask, acting as the central controller of the system. It handles user authentication (via JWT), request validation, image upload coordination, and metadata management.

When a user uploads an image, the backend stores the file in Amazon S3 and creates a processing job by sending a message to Amazon SQS. It also provides APIs for retrieving image metadata, classification results, and map data.

The backend serves as the integration layer connecting the frontend with storage systems and asynchronous services.

Asynchronous Processing Layer (SQS + Lambda)

To ensure responsiveness, time-consuming tasks are processed asynchronously. When a processing task is triggered (e.g., after an image upload or a batch download request), a message is placed into an SQS queue. AWS Lambda functions consume these messages and perform operations such as:

- Image classification using a machine learning model
- Metadata extraction (e.g., EXIF/location data)
- Follow-up question generation
- ZIP file processing, including extraction of uploaded archives and creation of ZIP files for batch downloads

This design decouples user requests from heavy computation, preventing blocking and improving system scalability.

Data Storage Layer (S3 + DynamoDB)

The system uses two main storage services:

- **Amazon S3** stores user-uploaded images, uploaded and generated ZIP files in one bucket, and hosts frontend static assets in a separate bucket.
- **Amazon DynamoDB** stores structured metadata such as user accounts, image labels, locations, follow-up questions and answers, processing status, and job records for ZIP-related tasks.

This separation of file storage and metadata storage allows efficient handling of large media files and flexible querying of structured data.

Machine Learning and External Services

The machine learning component is responsible for image classification. A pre-trained YOLOv8 model is used to predict image categories (e.g., person, animal, plant).

The system also integrates external map services such as OpenStreetMap and Leaflet to visualize image locations. The frontend retrieves location data from the backend and renders it on an interactive map.

Data Flow and Communication

The system follows three related but distinct data flows.

For **single image upload**, the user submits an image through the frontend, and the request is sent to the Flask backend. The backend stores the image in Amazon S3 and sends a message to SQS. AWS Lambda then consumes the message and performs image classification, metadata extraction, and follow-up question generation. The results are stored in DynamoDB, and the frontend queries the backend to retrieve the updated status and display the processed image information.

For **ZIP image upload**, the user uploads a ZIP archive through the frontend. The backend sends the ZIP processing task to SQS. A Lambda function consumes the task, extracts the archive, and groups the images into a list, which is then sent as a single message to the image-processing queue. The image-processing Lambda function supports batch processing and handles the list of images in one execution, following the same workflow as single image uploads.

For **ZIP download**, the user requests selected images to be downloaded as a ZIP file. The backend sends a compression task to SQS, and Lambda handles the file packaging process. The generated ZIP file is stored in Amazon S3, and its reference (e.g., file key or URL) is stored in DynamoDB. The frontend then retrieves the download link through the backend and accesses the file from S3.

This event-driven design keeps file processing and machine learning tasks separate from user-facing requests, improving responsiveness and allowing each workflow to be handled independently. Each SQS queue is connected to a specific Lambda function responsible for handling the corresponding task, forming a clear event-driven processing pipeline.

Design Considerations

The architecture emphasizes:

- **Decoupling:** Frontend, backend, and processing layers operate independently
- **Scalability:** Cloud services (S3, Lambda, DynamoDB) scale automatically
- **Responsiveness:** Asynchronous processing avoids blocking user requests
- **Reliability:** Failures are handled through status tracking and retry mechanisms

Technologies and Frameworks

The AI Cloud Album uses a set of lightweight and cloud-native technologies to support image upload, asynchronous processing, metadata storage, and map-based browsing. The main technologies are:

Programming Languages

- **Python**: used for backend development, AWS integration, asynchronous task handling, and machine learning inference
- **HTML and CSS**: used to build the frontend user interface

Frameworks

- **Flask**: used as the backend framework for implementing REST APIs, user authentication, and coordination with AWS services
- **YOLOv8**: used as a pre-trained image classification model for inference

AWS Services

- **Amazon EC2**: used to host the Flask backend server
- **Amazon S3**: used for file storage, with one bucket storing user-uploaded images, uploaded and generated ZIP files, and a separate bucket hosting frontend static assets
- **Amazon SQS**: used to queue asynchronous processing tasks
- **AWS Lambda**: used to process queued tasks such as image classification, metadata extraction, ZIP extraction, and ZIP creation
- **Amazon DynamoDB**: used to store structured data, including user information, image metadata, processing status, and job records for ZIP-related tasks
- **AWS IAM**: used to manage access control between AWS services by assigning appropriate roles and permissions

- **Amazon CloudWatch**: used for logging and monitoring Lambda execution and system behavior

Third-Party Services

- **OpenStreetMap / Leaflet**: used for map-based visualization of image locations
- **ImageNet-1k**: used as a reference dataset for model development and testing

Technology Justification

The technology stack was selected to match the system requirements and project scope. Flask was chosen for its simplicity and flexibility in building REST APIs and integrating with AWS services. S3 was used to store both user-uploaded files and frontend static assets, providing a unified and scalable storage solution. DynamoDB was selected because it efficiently handles semi-structured data such as image metadata and job tracking without requiring complex joins.

SQS and Lambda were used to implement an event-driven asynchronous processing architecture, ensuring that time-consuming tasks such as image processing and file operations do not block user requests. The use of batch processing in Lambda further improves efficiency for ZIP-based workflows. IAM roles were used instead of manual credential management to securely control access between services. CloudWatch provides visibility into system behavior and helps with debugging and monitoring.

YOLOv8 was chosen as a practical pre-trained model that can be easily integrated into the system without requiring extensive training. OpenStreetMap and Leaflet were selected to provide map visualization without relying on paid services.

AWS Deployment

The AI Cloud Album is deployed on AWS using a service-oriented architecture. The backend is hosted on an Amazon EC2 instance (t3.micro) running Ubuntu 24.04 (Linux/UNIX), using an official Ubuntu AMI. This lightweight instance type is sufficient for development and testing purposes.

The system uses Amazon S3 for file storage, with one bucket storing user-uploaded images and both uploaded and generated ZIP files, and a separate bucket hosting frontend static assets.

For data storage, the system uses Amazon DynamoDB, which stores structured metadata such as user accounts, image labels, locations, follow-up questions and answers, processing status, and job records for ZIP-related tasks.

Asynchronous processing is implemented using Amazon SQS and AWS Lambda. The backend sends task messages to SQS, and Lambda functions process them to perform image classification, metadata extraction, ZIP extraction, and ZIP creation.

Access control between services is managed using AWS IAM roles and policies, ensuring secure interaction between components. Amazon CloudWatch is used for logging and monitoring system behavior, especially for Lambda execution.

REST API Design

The system exposes a set of RESTful APIs to support user authentication, image management, asynchronous job tracking, and utility operations. All APIs follow a consistent structure and use JSON as the primary data format.

Authentication

Authentication is implemented using JWT (JSON Web Tokens). Protected endpoints require a Bearer token in the HTTP Authorization header:

```
Authorization: Bearer <your_token>
```

This mechanism ensures that only authenticated users can access and manage their own data.

API Structure and Data Format

All API responses follow a unified JSON structure:

- **Success response**

```
{
  "success": true,
  "message": "ok",
  "data": {}
}
```

- **Error response**

```
{
  "success": false,
  "message": "error message"
}
```

APIs use:

- `application/json` for standard requests
- `multipart/form-data` for file uploads

API Categories

The API endpoints are organized into several functional groups:

- **Authentication APIs** (`/api/auth/*`)
    - User registration and login
- **Image APIs** (`/api/images/*`)
    - Image upload (single and ZIP)
    - Image retrieval and listing
    - Follow-up interaction
    - Search and deletion
- **Job APIs** (`/api/jobs/*`)
    - Track asynchronous processing tasks such as ZIP upload and download
- **Collection and Map APIs**
    - Group images by label or location (`/api/collections/`)
    - Provide coordinates for map visualization (`/api/map/points`)
- **Utility APIs**
    - Generate presigned URLs for secure access to S3 files (`/api/utils/presigned-url`)

Example Endpoint

For example, the image upload API:

`POST /api/images/upload`

- Accepts file upload via `multipart/form-data`
- Stores the file in S3

- Returns an image ID, processing status, and the S3 key

Error Handling

The API uses standard HTTP status codes, including:

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 409 Conflict
- 413 Payload Too Large
- 500 Internal Server Error

Appendix Reference

Due to the number of endpoints and detailed request/response formats, the complete API documentation is provided in the Appendix.

Database Schema

The AI Cloud Album uses Amazon DynamoDB as its primary database for storing structured metadata and asynchronous job records. The schema is designed around the application's main workflows: user authentication, image upload and classification, follow-up metadata collection, and ZIP-related batch processing. Instead of using a relational schema with joins, the system stores each functional entity as a separate DynamoDB item type, which better matches the project's access patterns and cloud-native design.

Users

The `Users` table stores account and authentication information:

```
{
  "user_id": "xxx@example.com",        // Partition Key
  "email": "xxx@example.com",
  "password_hash": "...",
  "created_at": "2026-05-02T08:44:56.486718"
}
```

This table supports user registration and login. Passwords are stored as hashed values rather than plain text for security. The `user_id` is typically the user's email.

ImageMetadata

The `ImageMetadata` table stores metadata for each uploaded image:

```
{
  "user_id": "xxx@example.com",        // Partition Key
  "image_id": "123",                   // Sort Key

  "s3_key": "xxx@example.com/abc.jpg",
  "status": "uploaded | processing | needs_followup | complete | failed | deleted",

  "label": "cat",
  "confidence": 0.92,

  "location": {
    "lat": 34.02,
    "lng": -118.28
  },

  "followup_questions": ["Is this your pet?"],
  "followup_answers": ["Yes"],

  "created_at": "2026-05-02T08:44:56.486718",
  "updated_at": "2026-05-02T08:44:56.486718"
}
```

This table is the core of the image management workflow. It supports image upload tracking, classification results, location-based browsing, follow-up interaction, and status polling. The combination of `user_id` and `image_id` works well as a partition-and-sort key design because the system frequently queries images by user and then by image ID.

The actual image file is stored in Amazon S3, while DynamoDB stores only the metadata and the corresponding `s3_key`. The `confidence` field is retained from the initial design but is currently not used in the application logic and is typically set to `null`.

ZipJobs

The `ZipJobs` table stores asynchronous job records for ZIP upload and ZIP download operations:

```
{
  "user_id": "xxx@example.com",        // Partition Key
  "job_id": "456",                     // Sort Key

  "type": "zip_upload | download",

  "status": "uploaded | processing | complete | failed",

  "s3_key": "uploads/zips/xxx.zip",  // zip_upload only
  "result_s3_key": "downloads/xxx.zip",  // download only

  "total_files": 10,
  "processed_files": 3,

  "image_ids": ["img1", "img2", "img3", "img4", "img5"],  // download only

  "created_at": "2026-05-02T08:44:56.486718",
  "updated_at": "2026-05-02T08:44:56.486718"
}
```

This table is used to track long-running batch workflows. ZIP upload jobs store the uploaded archive location and processing status, while ZIP download jobs store the selected image list and the final ZIP output location. A separate job table makes asynchronous processing easier to manage and keeps task state independent from image metadata.

The `processed_files` field was included in the initial schema design for batch progress tracking, but it is not currently used in the application logic.

Design Decisions

Several design decisions were made to match the application requirements:

1. **Use DynamoDB instead of RDS**
   The system mainly stores semi-structured metadata and status records, not relational data with complex joins. DynamoDB is a better fit for flexible, scalable key-value and document-style access patterns.

2. **Store files in S3, not in the database**
   Images and ZIP files are stored in Amazon S3, while DynamoDB stores only file references such as `s3_key`. This keeps the database lightweight and avoids storing large binary objects.

3. **Separate user, image, and job data**
   User accounts, image metadata, and ZIP job records have different access patterns and lifecycles. Separating them improves clarity and maintainability.

4. **Embed small related metadata in image records**
   Fields such as `location`, `followup_questions`, and `followup_answers` are stored directly in the image record because they are naturally tied to a single image and do not require separate tables.

5. **Track processing state explicitly**
   The `status` field is important for asynchronous workflows. It allows the frontend to poll for updates and show whether an image or job is `uploaded`, `processing`, `complete`, or `failed`.

Overall, the schema supports the main application features: user authentication, image upload, ML classification, follow-up interaction, ZIP processing, and map-based browsing.

Authentication and User Roles

Authentication Mechanism

The system uses JWT (JSON Web Token) for user authentication. When a user registers or logs in through the authentication APIs, the backend generates a JWT token and returns it to the client. The client stores this token and includes it in the `Authorization` header of subsequent requests:

```
Authorization: Bearer <your_token>
```

The backend verifies the token for each protected API request. If the token is missing or invalid, the request is rejected with an appropriate HTTP error (e.g., 401 Unauthorized). This stateless authentication mechanism simplifies session management and allows the system to scale without maintaining server-side session data.

User Roles

The system adopts a single-role model, where all users have the same role and permissions. There are no role-based access control (RBAC) distinctions such as admin or guest users.

Each user can:

- Upload and manage their own images
- View classification results and metadata
- Submit follow-up information
- Perform ZIP upload and download operations

This simplified role design is sufficient for the project scope and avoids unnecessary complexity.

User-Specific Content Management

User-specific content is managed through the `user_id` field, which is included in all major data entities such as image metadata and job records.

- In the **ImageMetadata** table, `user_id` is used as the partition key, ensuring that images are logically grouped by user.
- In the **ZipJobs** table, `user_id` is also used as the partition key to track user-specific asynchronous tasks.

When a request is received, the backend extracts the `user_id` from the JWT token and ensures that the user can only access their own resources. For example:

- A user can only retrieve images associated with their `user_id`
- Access to other users' images or jobs is denied

This design enforces data isolation and ensures that user data remains private and secure.

External Dependencies

The AI Cloud Album depends on several external libraries, frameworks, AWS services, and third-party resources to support backend processing, frontend interaction, machine learning inference, and cloud deployment.

Backend Dependencies (Python)

The backend is implemented in Python using Flask and relies on common libraries for web development, authentication, and AWS integration. These dependencies are typically managed through a `requirements.txt` file, which includes the following key packages:

- **Flask (3.0.3)**: provides the core web framework for building REST APIs
- **boto3 (1.34.162)**: AWS SDK for interacting with services such as S3, SQS, and DynamoDB
- **PyJWT (2.9.0)**: used for generating and verifying JWT tokens for authentication
- **Werkzeug (3.0.3)**: used for request handling and password hashing utilities
- **python-dotenv (1.0.1)**: used for managing environment variables during development
- **gunicorn (25.3.0)**: WSGI HTTP server used to run the Flask application in a production-like environment
- **flask-cors (4.0.0)**: enables Cross-Origin Resource Sharing (CORS) to allow frontend-backend communication

These dependencies support core backend functionalities including API handling, authentication, AWS integration, and deployment.

Machine Learning Dependencies

The system uses a pre-trained model for image classification:

- **YOLOv8 (Ultralytics)**: used for image classification inference
- **PyTorch(training only, not for run time)**: underlying deep learning framework used by YOLOv8
- **Onnxruntime:** For ML running in lambda.
- **PIL:** For processing Images.
- **Zipfile**: For zip files.

For Lambda(including machine learning), dependencies are packaged in ZIP file, no special requirements for deployment.

These libraries are used in the asynchronous processing layer (Lambda or backend environment) to generate classification results.

AWS Services

The system heavily relies on AWS managed services:

- **Amazon EC2**: backend hosting
- **Amazon S3**: file storage (images, ZIP files, frontend assets)
- **Amazon SQS**: message queue for asynchronous processing
- **AWS Lambda**: background processing for ML and file operations
- **Amazon DynamoDB**: metadata and job storage
- **AWS IAM**: access control
- **Amazon CloudWatch**: logging and monitoring

Frontend Dependencies

The frontend is implemented using:

- **HTML and CSS**: for structure and styling
- **JavaScript (vanilla)**: for interacting with backend APIs

No heavy frontend frameworks are used, which keeps the system simple and lightweight.

Third-Party Services

The system integrates external services for additional functionality:

- **OpenStreetMap (OSM)**: provides map tile data
- **Leaflet.js**: used for rendering interactive maps in the frontend

These services enable visualization of image locations without relying on paid map APIs

External APIs

The system primarily uses its own backend APIs. It does not rely on external REST APIs for core functionality, except for map tile services provided by OpenStreetMap.

User Experience

The AI Cloud Album system provides a web-based interface that guides users through a complete workflow from account creation to image management and retrieval.

A typical user interaction begins with the login/register page, where users can switch between registration and login modes. During registration, the system validates the email format to ensure correctness. After successfully creating an account, users can log in using their credentials and are redirected to the main page.

Once logged in, users arrive at the main page, which displays all images they have uploaded. This page serves as the central hub of the application. Users can search for images using the search bar by entering keywords such as labels or user-defined names. They can also select and delete unwanted images. The interface provides navigation to other pages through a sidebar and interactive image grids.

From the main page, users can access the upload functionality, where they can upload either a single image or a ZIP file containing multiple images. Before submitting, users can review and remove files from the upload list. After initiating the upload, the system provides feedback indicating whether the operation was successful. The uploaded images are then processed asynchronously in the background.

Users can navigate to the detail page of an image to view additional information. This page displays metadata such as classification results and geographic location on a small map. If applicable, the system presents follow-up questions, allowing users to provide additional context (e.g., naming a person or object in the image). This enhances the metadata and improves future search results.

The map view page provides a global perspective by displaying all uploaded images on an interactive map. Each image is represented as a marker. When users hover over a marker, a preview of the image is shown, and they can navigate directly to the image's detail page.

For downloading, users can access the download page, where they select multiple images. After submitting the request, the system processes the images asynchronously, packages them into a ZIP file, and provides a download link once the task is complete. The system also notifies users whether the download process was successful.

Overall, the system offers a smooth and intuitive workflow that combines asynchronous processing with interactive user interfaces, allowing users to efficiently manage and explore their image collections.

Implementation Results

Completed Features

We successfully implemented the core functionality of the AI Cloud Album system. The deployed system supports the following features:

- **User authentication**: users can register and log in using JWT-based authentication
- **Image upload**: users can upload single images or ZIP files containing multiple images
- **Asynchronous processing**: uploaded images are processed using an event-driven pipeline (SQS + Lambda)
- **Automatic image classification**: images are classified using a machine learning model (YOLOv8 / ONNX)
- **Metadata management**: classification results, location data, and user-provided information are stored in DynamoDB
- **Follow-up interaction**: the system generates follow-up questions and allows users to submit additional metadata
- **Image browsing and search**: users can search images by labels or user-provided keywords
- **ZIP download**: users can select images and download them as a ZIP file
- **Map visualization**: image locations can be displayed on an interactive map

These features form a complete end-to-end workflow from image upload to retrieval and visualization.

Incomplete or In-Progress Features

Some features were designed but not fully implemented:

- **Dynamic follow-up question system**

  The system was initially designed to generate different follow-up questions based on classification results and use user input to improve grouping and search. However, this functionality is currently only implemented for the animal category with a single predefined question. As a result, the system lacks flexibility in handling different types of images. Completing this feature would require a more dynamic question generation mechanism and better integration with classification outputs.

- **Collections**

  The backend API for collections (grouping images by label or location) has been fully implemented. However, the corresponding frontend interface is not yet completed. Currently, users can still achieve similar functionality through search, but the dedicated collection view is not available.

- **Batch progress tracking**

  The system includes database fields (e.g., `processed_files`) intended for tracking batch processing progress. However, this feature is not fully implemented in the current version, and progress is not actively updated or displayed to users. Completing this feature would require updating the processing logic and adding frontend support for progress visualization.

Technical Challenges and Solutions

During the development of the AI Cloud Album, several technical challenges were encountered across system integration, asynchronous processing, and cloud deployment. The following summarizes key challenges, how they were diagnosed, and the solutions applied.

Integration of Multiple AWS Services

One major challenge was coordinating multiple AWS services, including S3, SQS, Lambda, and DynamoDB. Each service has its own data format, permissions, and interaction patterns, making integration complex.

To address this, we defined clear data contracts between components, such as consistent message formats for SQS and standardized metadata structures in DynamoDB. We also used IAM roles to manage permissions between services, reducing configuration errors.

Asynchronous Workflow Design

Designing a reliable asynchronous processing pipeline was another challenge. Tasks such as image classification, ZIP extraction, and ZIP creation needed to be decoupled from user requests while still allowing users to track progress.

We solved this by introducing an event-driven architecture using SQS and Lambda. Each task is represented as a message, and processing results are stored with explicit status fields (e.g., `processing`, `complete`). The frontend periodically polls the backend to retrieve updated status, ensuring a responsive user experience.

Handling ZIP File Processing

ZIP upload and download workflows introduced additional complexity, especially in managing batch operations and large file handling.

For ZIP uploads, the system extracts the archive and processes images in batches using Lambda. For downloads, selected images are packaged into a ZIP file asynchronously and stored in S3. Using a separate job table in DynamoDB helped track these long-running tasks.

Debugging and Monitoring in Cloud Environment

Debugging distributed systems in AWS was more difficult compared to local development. Errors in Lambda execution or SQS message handling were not always immediately visible.

We addressed this by using CloudWatch logs to monitor Lambda execution and track errors. Logging was added at key points in the workflow (e.g., message processing, database updates) to help identify failures and trace system behavior.

Authentication and Access Control

Ensuring that users can only access their own data was another important challenge. Since the system is stateless and uses JWT, proper validation is required for each request.

We implemented JWT-based authentication and extracted the `user_id` from tokens for each request. Backend APIs enforce access control by checking ownership of resources (e.g., images and jobs), ensuring data isolation between users.

Cors and Frontend Backend Integration

We encountered CORS problems because frontend files were stored in different S3 Buckets from backend files. We introduced a presigned URL to locate the real position of the images in the backend S3 Bucket.

Remaining Limitations and Future Improvements

Some features were designed but not fully implemented due to time constraints. For example, fields such as `confidence` (image classification score) and `processed_files` (job progress tracking) were included in the schema but are not actively used in the current system.

With more time, we would:

- Implement progress tracking for batch jobs
- Utilize confidence scores for filtering or ranking results
- Improve error handling and retry mechanisms
- Enable HTTPS and stronger security configurations for production deployment
- Add collection recommendations for albums
- Implement search and collection based on city or state names

Critical Reflection

Initial Misunderstandings and Learning

Before starting this project, we underestimated the complexity of building a cloud-based system. We initially assumed that integrating multiple AWS services would be straightforward once each component worked individually. However, during implementation, we realized that the main challenge lies not in individual components, but in how they interact within a distributed system.

For example, designing asynchronous workflows required careful coordination between SQS, Lambda, and DynamoDB. Issues such as message format consistency, state tracking, and failure handling were not obvious at the design stage but became critical during implementation. This experience changed our perspective: a cloud system is not simply a collection of services, but a coordinated system where data flow and state management are essential.

In addition, we initially considered enabling direct communication between AWS Lambda and the backend to return processing results immediately. However, we found that this approach was difficult to implement reliably in an asynchronous and distributed environment. Instead, we redesigned the workflow to use a status-based tracking mechanism, where each image or job record includes a `status` field (e.g., `processing`, `complete`). Lambda updates the status, and the backend retrieves results by querying the database. This approach proved to be simpler and more robust.

In the end, we oversimplified the integration of frontend page designs and backend APIs. The locally deployed frontend design had lots of mismatches with the API design, and faced lots of communication problems when deployed to the AWS.

This experience highlighted an important lesson: in distributed systems, indirect communication through shared state (such as database status fields) is often more reliable than direct communication between components.

Critical Technical Decisions

One of the most important decisions was adopting an event-driven architecture using SQS and Lambda instead of handling all tasks synchronously in the backend. This significantly improved system responsiveness and scalability, but also introduced complexity in debugging and state management.

Another key decision was using DynamoDB instead of a relational database. Since the system primarily performs queries based on `user_id` and `image_id`, a NoSQL database provides a more suitable and scalable solution. However, this required us to rethink data modeling, as we could not rely on joins or relational constraints.

We also chose to separate file storage (S3) from metadata storage (DynamoDB). This design improved efficiency and scalability, but required careful coordination to maintain consistency between file references and database records.

Understanding Integration and Deployment

Through this project, we developed a deeper understanding of system integration and deployment. Initially, we focused on implementing individual features such as upload, classification, and database operations in isolation. However, during deployment, we realized that making these components work together reliably is significantly more challenging.

Ensuring consistency across S3 storage, SQS messages, Lambda processing, and DynamoDB updates required careful design of data flow and status management. We also learned the importance of observability tools such as CloudWatch, which are essential for debugging distributed systems and understanding system behavior.

Another important realization was that deployment is not simply about running code on EC2, but about coordinating multiple managed services, configuring IAM permissions, and ensuring secure and reliable communication between components.

Overall, this project shifted our understanding from building standalone applications to designing integrated, cloud-based distributed systems.

Milestones, Timeline, and Contributions

Timeline & Milestones

The development of the AI Cloud Album system followed a structured timeline with clearly defined milestones:

- **By April 12**:
  The overall system design was completed, including frontend, backend, and machine learning components.
- **By April 19**:
  The majority of frontend interfaces were implemented and tested locally.
  The backend completed most API endpoints and initial integration with AWS services, including S3, DynamoDB, and SQS.
  The machine learning component completed local model setup and the design of the asynchronous processing pipeline.
- **By April 26**:
  Most system components were independently deployed and tested in local environments.
  Integration of the frontend, backend, and Lambda components began, and deployment

to AWS was initiated.
Only minor adjustments to the system design were required at this stage.

- **By April 28**:
  All components were successfully integrated and deployed on AWS.
  End-to-end system functionality was tested, with only a few remaining bugs identified.
- **By April 29**:
  The system was finalized, and all major features were completed.

Team Responsibilities

- **Boxiang Zhang (Frontend + UI Integration)**

  Responsible for developing the HTML and CSS frontend, including user authentication pages, image upload interface, album browsing, map visualization, and dynamic follow-up forms. Also handles frontend-backend integration and status polling for asynchronous tasks.

- **Huaitang Wang (Backend + AWS Integration)**

  Responsible for implementing the Flask REST API, including user authentication, image upload, and metadata management. Integrates the backend with AWS services such as S3, SQS, and DynamoDB, and coordinates communication with Lambda. Also supports asynchronous workflow design, deployment on EC2, and debugging of distributed system integration.

- **Shi He (ML + Asynchronous Processing)**

  Responsible for integrating the YOLOv8 model, implementing image classification, metadata extraction (EXIF/location), and follow-up question generation. Also develops Lambda functions to process SQS messages and write results to the database.